

Mixture-of-Experts Knowledge Graph Retrieval-Augmented Generation for Multi-Agent LLM-based Recommendation

Shijie Wang
shijie.wang@connect.polyu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Chengyi Liu
chengyi.liu@connect.polyu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Yujuan Ding✉
dingyujuan385@gmail.com
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Shanru Lin
llam32316@gmail.com
The Hong Kong Polytechnic
University
Hong Kong SAR, China

See-Kiong Ng
seekiong@nus.edu.sg
National University of Singapore
Singapore, Singapore

Xu Xin
xin.xu@polyu.edu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Wenqi Fan
wenqifan03@gmail.com
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Abstract

Large language models (LLMs) have recently been adopted for recommendations due to their ability to understand user intent and item semantics. However, LLM-based recommender systems often rely on parametric knowledge and suffer from outdated knowledge, motivating knowledge graph retrieval-augmented generation (KG-RAG) to ground recommendations on structured, up-to-date KGs. Despite this promise, effective KG-RAG in recommendations faces great challenges. First, users' queries vary in complexity and require KG knowledge at different granularities, whereas existing methods adopt a one-size-fits-all retrieval strategy, leading to over-retrieval for simple queries and under-retrieval for complex ones. In addition, augmenting LLMs with KG knowledge requires translating graph-structured data into linear text, which may introduce noise and cause structural information loss. Moreover, the selection of retrieval granularity lacks direct supervision and must be inferred from the final recommendation after alignment and downstream utilization, making query-aware retrieval hard to learn end-to-end. To address these issues, we propose *MixRAGRec*, a cooperative multi-agent framework for KG-RAG recommendations. *MixRAGRec* integrates a Mixture-of-Experts Retrieval Agent that routes each query to a KG retrieval expert with different granularities, a Knowledge Preference Alignment Agent that converts structured knowledge

into LLM-friendly natural language, and a Contrastive Learning-reinforced Recommendation Agent trained with contrastive preference feedback. Notably, we introduce Mixture-of-Experts Multi-Agent Policy Optimization (MMAPO) to train three agents under a unified objective. Extensive experiments on real-world datasets demonstrate the effectiveness of our framework. Our codes are available at <https://github.com/Sjay-Wang/MixRAGRec>.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Large Language Models, Recommendations, Retrieval-augmented Generation, Multi-agent Reinforcement Learning

ACM Reference Format:

Shijie Wang, Chengyi Liu, Yujuan Ding, Shanru Lin, See-Kiong Ng, Xu Xin, and Wenqi Fan. 2026. Mixture-of-Experts Knowledge Graph Retrieval-Augmented Generation for Multi-Agent LLM-based Recommendation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3770855.3817630>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.20372864>.

✉ Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD 2026, Jeju Island, Republic of Korea.*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817630>

1 Introduction

Recommender systems, as a key technique for alleviating information overload and enabling personalized decision-making, have attracted increasing attention across domains such as e-commerce and social media [5, 8, 35, 37]. Recently, Large Language Models (LLMs) have achieved significant breakthroughs and show strong potential as a new paradigm for recommender systems [2, 29, 47].

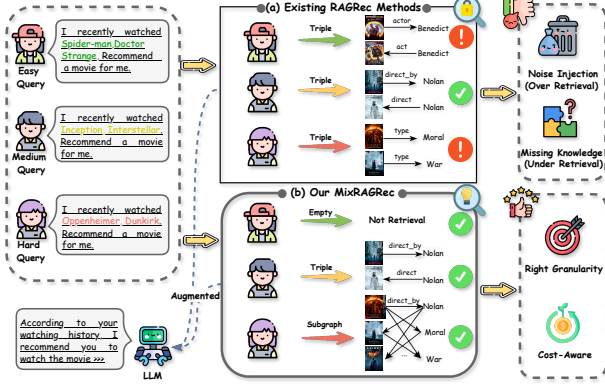


Figure 1: Illustration of existing RAGRec methods and our proposed MixRAGRec methods. (a) Existing RAGRec methods apply the same KG retrieval granularity to all queries, causing either noise from over-retrieval or missing knowledge from under-retrieval; (b) Our MixRAGRec routes queries of different complexity to appropriate retrieval granularity, improving the utility-cost trade-off.

By training on vast and diverse data, LLMs exhibit strong generalization and nuanced language understanding capabilities [4, 28, 30, 40]. These capabilities enable LLM-based recommender systems to distill user preferences from profiles, item metadata, and interaction histories for more tailored suggestions [34, 43].

Despite their remarkable advantages in language understanding and generalization, existing LLM-based recommender systems still suffer from critical limitations, including hallucinations and insufficient up-to-date, domain-specific knowledge [23, 36]. For instance, an LLM may recommend a non-existent sequel like “*The Matrix 5: Reawakened*” to a fan of the “*Matrix*”. Furthermore, the static nature of LLMs’ training data often leaves them unable to recommend the latest items absent from the pre-training corpus. To alleviate these issues, recent Retrieval-Augmented Generation (RAG) leveraging external Knowledge Graph (KG) has emerged as a promising solution to provide the latest and domain-specific structured knowledge without costly fine-tuning [33, 36]. By grounding LLMs in *external, structured, updated, and editable* knowledge sources, KG-RAG provides a robust mechanism for supplying factual, up-to-date, and domain-specific information, thereby mitigating hallucinations and knowledge latency.

However, achieving effective KG-RAG recommender systems faces significant challenges. *First*, user queries require KG knowledge at different granularities, while existing KG-RAG recommender systems typically adopt a one-size-fits-all retrieval granularity [16, 36]. This mismatch leads to over-retrieval for simple queries (introducing redundant context and extra cost) and under-retrieval for complex queries (missing multi-hop relations needed for reasoning), as illustrated in Figure 1(a). Hence, it remains challenging to design a query-aware retrieval mechanism that selects an appropriate retrieval granularity to balance knowledge utility and retrieval cost. *Additionally*, the retrieved knowledge is naturally graph-structured, while the input to LLMs is plain text. Directly describing structured knowledge into text can introduce redundant noise and lose structural information, reducing the utility of retrieved knowledge [32].

Thus, it is challenging to convert KG knowledge into an LLM-friendly representation that preserves recommendation-relevant knowledge while avoiding unnecessary noise. *Furthermore*, the selection of retrieval granularity lacks direct supervision and can only be inferred from the final recommendation outcome after alignment and downstream utilization. This intricate interlinking makes the influence of retrieval granularity hard to disentangle, while applying vanilla end-to-end optimization with only the recommendation criteria may converge to a fixed granularity preference. This poses a special challenge to effectively coordinate the retrieval, alignment, and recommendation under a unified training objective.

To address these challenges, we propose **MixRAGRec** (Mixture-of-Experts KG-RAG for LLM-based Recommendation), a cooperative multi-agent KG-RAG framework that integrates query-aware retrieval, knowledge alignment, and knowledge-augmented recommendation. Specifically, MixRAGRec consists of three cooperating agents: (1) a **Mixture-of-Experts Retrieval Agent** that routes each query to one of multiple KG retrieval experts with different granularities, ranging from *no retrieval* (direct generation) to triple retrieval, neighborhood subgraph retrieval, and connected-graph retrieval; (2) a **Knowledge Preference Alignment Agent** that transforms retrieved structured knowledge into an LLM-friendly natural-language knowledge snippet, bridging the gap between graph-structured knowledge and textual inputs; and (3) a **Contrastive Learning-reinforced Recommendation Agent** trained with **contrastive preference feedback**, which encourages the model to rank the target item above competitive negatives under the same context. To jointly optimize these agents, we introduce **MMAPO** (Mixture-of-Experts Multi-Agent Policy Optimization), which coordinates discrete expert selection, knowledge alignment, and downstream recommendation under a shared objective. In particular, MMAPO incorporates a **marginal information gain** term that quantifies retrieval utility relative to its computational cost, encouraging the system to retrieve richer knowledge only when the expected benefit justifies the added overhead. The main contributions of this paper can be summarized as follows:

- We propose MixRAGRec, a multi-agent KG-RAG framework for LLM-based recommendation that performs query-aware retrieval with different granularities via a mixture of KG experts.
- We introduce a Knowledge Preference Alignment agent that converts structured KG knowledge into concise, LLM-friendly natural language, mitigating the structure-semantics gap.
- We propose MMAPO, a unified optimization framework that couples agents with a shared objective and a retrieval-utility reward for cost-aware retrieval.
- We conduct comprehensive experiments on various real-world datasets to demonstrate the effectiveness of the proposed MixRAGRec framework.

2 Related Work

Retrieval-Augmented Generation (RAG) has emerged as a robust strategy to mitigate LLM limitations, such as hallucinations and lack of knowledge, by grounding outputs in external knowledge sources [7, 9, 11, 20]. Early studies such as RETRO [3] and FLARE [18] typically retrieve fragments from unstructured corpora (e.g., passages or documents) and condition generation on the retrieved

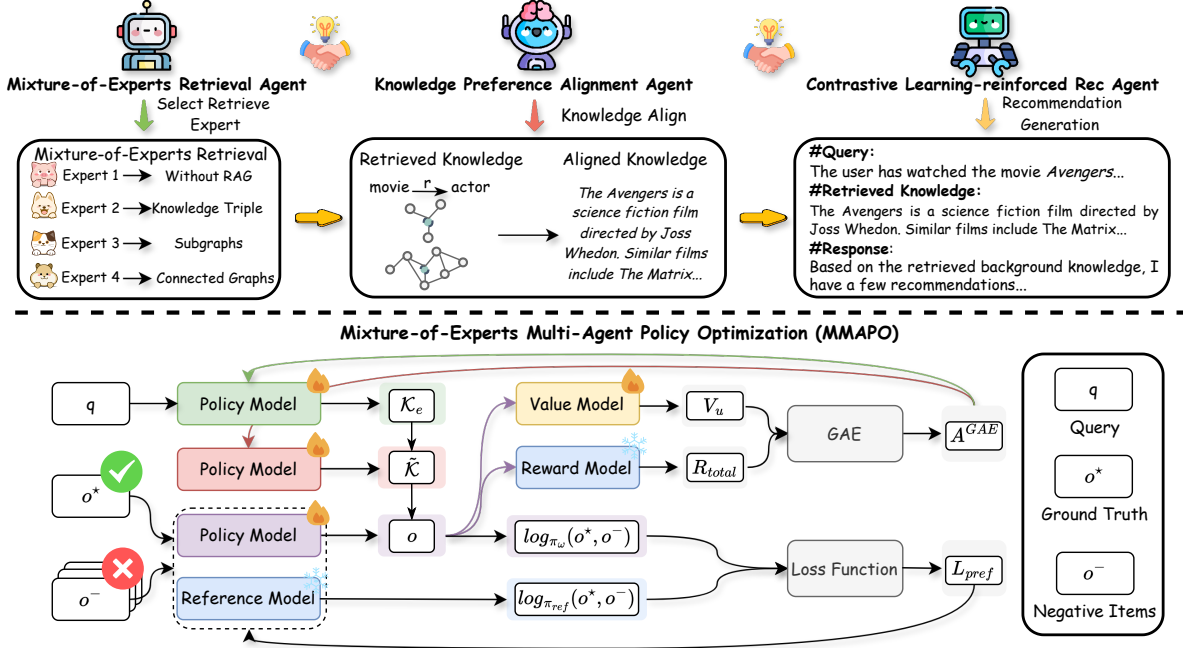


Figure 2: The overview of the MixRAGRec, which consists of three agents: (1) *Mixture-of-Experts Retrieval Agent* selects a retrieval expert to obtain structured knowledge, (2) *Knowledge Preference Alignment Agent* converts the retrieved KG knowledge into an LLM-friendly representation, and (3) *Contrastive Learning-reinforced Recommendation Agent* produces the recommendation. *Mixture-of-Experts Multi-Agent Policy Optimization (MMAPO)* jointly optimizes these agents under a shared objective.

context. Beyond text corpora, recent studies explore augmenting LLMs with graph-structured knowledge (e.g., KGs) to provide structured knowledge for LLM inference [13, 14, 27, 38]. Representative studies such as G-retriever [14] and GNN-RAG [27] retrieve subgraphs from KGs and incorporate them into LLM inference via either structured encoders or text-based serialization.

Building on these advances, RAG has been introduced into recommender systems to ground LLM-based recommendations on external knowledge, mitigating issues such as hallucinations and outdated knowledge. Depending on the source of retrieved knowledge, existing RAG-based recommender systems can be broadly grouped into two types. The first type of methods typically retrieve knowledge from datasets (e.g., historical user-item interactions and user behaviors) to provide domain-specific knowledge for LLMs to improve user modeling [21, 23, 26, 41, 42, 49]. For instance, ReLLa [23] and RALLRec [41] retrieve the most relevant behaviors from long interaction histories instead of relying solely on the most recent actions, while G-Refer [21] retrieves user profiles and explanation paths from interaction graphs to enhance explainability. The other methods retrieve knowledge from external knowledge bases, most notably knowledge graphs, to provide structured and up-to-date knowledge for LLM-based recommendations [33, 36, 46]. A representative method, K-RagRec [36], retrieves relevant knowledge subgraphs for cold-start items to improve knowledge-augmented recommendation. Despite their success, existing methods are typically built on fixed retrieval and ignore the difficulties of queries, which can lead to suboptimal accuracy and efficiency. To fill this gap, we propose MixRAGRec to enable query-aware retrieval and effective utilization of retrieved KG knowledge within a unified framework. We provide more related work in Appendix F.

3 Methodology

In this section, we first introduce some key notations and concepts in this paper. Then, we provide the details for each component of our proposed framework MixRAGRec.

3.1 Problem Formulation

In this paper, we consider a knowledge-augmented recommendation scenario. For each instance, the system receives a user query q describing the user intent and profile context (e.g., historical interactions), and outputs a recommended item o from a candidate set O . To support retrieval-augmented recommendation, the system can access a KG $G = (\mathcal{V}, \mathcal{R}, \mathcal{T})$, where $\mathcal{V}$ is the node set, $\mathcal{R}$ is the relation set, and $\mathcal{T} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ is the triple set. Given a query q , the system may retrieve a knowledge set from G . We define our retrieval expert set with four experts, which is extendable to more. Specifically, $\mathcal{E}_{\text{exp}} = \{e_1, e_2, e_3, e_4\}$ and each e returns structured knowledge $\mathcal{K}_e$ at a specific granularity.

It is worth noting that MixRAGRec is formulated as a cooperative multi-agent decision process. Generally, given q , a Mixture-of-Experts retrieval agent chooses an expert $e \in \mathcal{E}_{\text{exp}}$ to execute retrieval; a knowledge preference alignment agent transforms the retrieved knowledge $\mathcal{K}_e$ into a natural-language representation $\tilde{\mathcal{K}}$; finally, a recommendation agent outputs o conditioned on $(q, \tilde{\mathcal{K}}, O)$. The framework aims to optimize recommendation performance while accounting for retrieval utility. Formally, the objective function is defined as:

$$(\theta^*, \psi^*, \omega^*) = \arg \max_{\theta, \psi, \omega} \mathbb{E}_{(q_i, O_i, o_i^*) \sim \mathcal{D}} \left[\mathbb{E}_{e_i \sim \pi_{\theta}(\cdot | s_i)} \left[\mathbb{E}_{\tilde{\mathcal{K}}_i \sim \pi_{\psi}(\cdot | q_i, \mathcal{K}_{i, e_i})} \mathbb{E}_{o_i \sim \pi_{\omega}(\cdot | q_i, \tilde{\mathcal{K}}_i, O_i)} [R(q_i, O_i, o_i^*; e_i, \tilde{\mathcal{K}}_i, o_i)] \right] \right], \quad (1)$$

where π_θ , π_ψ , and π_ω are the policies of the retrieval, alignment, and recommendation agents, respectively. $\mathcal{D}$ denotes the training distribution and $o^* \in \mathcal{O}$ is the ground-truth item. s_i is the retrieval state constructed from the query context, and $R(\cdot)$ is the reward function detailed in Section 3.6.2.

3.2 The Overview of MixRAGRec

As illustrated in Figure 2, MixRAGRec is a multi-agent framework for knowledge-augmented recommendation that consists of three crucial agents: the *Mixture-of-Experts Retrieval Agent*, the *Knowledge Preference Alignment Agent*, and the *Contrastive Learning-reinforced Recommendation Agent*. Specifically, given a user query, the Mixture-of-Experts retrieval agent selects the appropriate retrieval expert to obtain structured knowledge from the KG. Next, the knowledge preference alignment agent converts the retrieved knowledge into an LLM-friendly representation, bridging the representational gap between graph-structured knowledge and text-based modeling. Finally, the contrastive learning-reinforced recommendation agent produces the recommendation based on the query and aligned knowledge. To train these agents under a shared objective and enable query-aware expert selection, we propose *Mixture-of-Experts Multi-Agent Policy Optimization (MMAPO)*, which coordinates them through a shared reward that accounts for both recommendation performance and retrieval utility. This design enables MixRAGRec to generate accurate recommendations by adapting retrieval for diverse queries through a unified multi-agent framework.

3.3 Multi-Granularity KG Retrieval with Mixture-of-Experts

MixRAGRec adopts a Mixture-of-Experts retrieval module that retrieves structured knowledge from an item-centric KG at multiple granularities. As illustrated in Figure 3, the left part shows KG construction and indexing, while the right part depicts a Mixture-of-Experts retrieval design, ranging from direct generation without retrieval to triple-, subgraph-, and connected-graph retrieval. This design aims to provide query-relevant knowledge while avoiding unnecessary retrieval cost and noise.

3.3.1 KG Construction and Indexing. We represent the knowledge source as a KG $G = (\mathcal{V}, \mathcal{R}, \mathcal{T})$, where $\mathcal{V}$ and $\mathcal{R}$ denote the sets of entities and relations, and $\mathcal{T} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ denotes the set of triples (h, r, t) . To facilitate flexible retrieval, we encode both entities and triples into a shared embedding space using a pre-trained language encoder. For an entity $n \in \mathcal{V}$ with textual description x_n , we capture the semantic information as $\mathbf{z}_n = \text{Encode}(x_n) \in \mathbb{R}^d$, where d denotes the dimension of the output representation. Then we build an entity vector database $\mathcal{Z}_\mathcal{V} = \{\mathbf{z}_n\}_{n \in \mathcal{V}}$. Similarly, for a triple $(h, r, t) \in \mathcal{T}$ with text descriptions (x_h, x_r, x_t) , we compute

$$\mathbf{z}_{(h,r,t)} = \text{Encode}(\text{Concat}(x_h, x_r, x_t)) \in \mathbb{R}^d, \quad (2)$$

and construct a triple vector database $\mathcal{Z}_\mathcal{T} = \{\mathbf{z}_{(h,r,t)}\}_{(h,r,t) \in \mathcal{T}}$. By utilizing this dual indexing, retrieval experts can access KG knowledge at different levels of granularity.

3.3.2 Mixture-of-Experts KG Retrieval. While existing KG-RAG methods provide a general mechanism to incorporate KG knowledge, they typically rely on a fixed retrieval strategy. In practice,

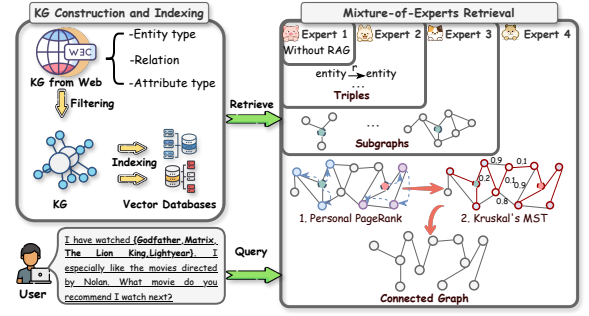


Figure 3: Demonstration of KG Construction and Indexing and the Retrieval Experts.

user queries vary widely, ranging from simple cases requiring little or no external knowledge to complex ones involving multi-hop knowledge subgraphs. As a result, a fixed KG retrieval strategy may under-retrieve for complex queries, leading to insufficient knowledge for accurate recommendation, or over-retrieve for simple queries, introducing unnecessary noise and computational overhead.

To this end, we propose a Mixture-of-Experts retrieval agent that selects the appropriate retrieval granularity for each query. Specifically, four retrieval experts are designed to address diverse query scenarios, including direct generation without retrieval, triple-level retrieval for precise facts, k -hop neighborhood subgraphs capturing higher-order relations, and connected subgraphs representing global knowledge, as shown in Figure 3.

Given a user query q , we compute its embedding using the same encoder as in indexing to ensure a consistent representation space: $\mathbf{z}_q = \text{Encode}(q) \in \mathbb{R}^d$.

The Mixture-of-Experts retrieval agent then selects one expert $e \in \{1, 2, 3, 4\}$ and returns the retrieved KG knowledge $\mathcal{K}_e$, as illustrated in Figure 3 (right).

Expert 1: DirectGenerator (no retrieval). This expert performs no KG retrieval and returns an empty set, i.e., $\mathcal{K}_1 = \emptyset$.

Expert 2: TripleRetriever (triple-level retrieval). This expert retrieves the most relevant KG triples by semantic similarity. Formally, the retrieved triple set is:

$$\mathcal{K}_2 = \text{TopM}_{(h,r,t) \in \mathcal{T}} \text{sim}(\mathbf{z}_q, \mathbf{z}_{(h,r,t)}), \quad (3)$$

where M is the retrieval budget, and is instantiated differently for different granularities. $\text{sim}(\cdot, \cdot)$ measures the cosine similarity.

Expert 3: SubgraphRetriever (k -hop neighborhood subgraph). To capture multi-hop relational context, this expert first selects a seed set of relevant entities:

$$\mathcal{S} = \text{TopM}_{n \in \mathcal{V}} \text{sim}(\mathbf{z}_q, \mathbf{z}_n), \quad (4)$$

and then expands each seed into its k -hop neighborhood subgraph. Denote the k -hop neighborhood subgraph around entity n as $G_n^{(k)}$, the retrieved knowledge is the union $\mathcal{K}_3 = \bigcup_{n \in \mathcal{S}} G_n^{(k)}$.

Expert 4: ConnectedGraphRetriever (connected subgraph). For complex queries that benefit from globally coherent knowledge, the expert constructs a connected subgraph that captures global knowledge. It begins with the same seed set $\mathcal{S}$ as Expert 3, and then executes a Personalized PageRank (PPR) [31] from $\mathcal{S}$ to obtain importance scores over entities. Then the top- M entities by PPR score, denoted $\mathcal{V}_{\text{PPR}}$, will be selected and consider the induced

subgraph $G[\mathcal{V}_{\text{PPR}}]$. To obtain a connected subgraph containing the most relevant entities while minimizing edge costs, we formulate the retrieval objective as finding a minimum-cost spanning tree (MST) over $G[\mathcal{V}_{\text{PPR}}]$. Specifically, we assign each edge e a cost based on the semantic match between the query and the relation type of the edge as $\text{cost}(e) = 1 - \cos(\mathbf{z}_q, \mathbf{z}_{r(e)})$, where $\mathbf{z}_q$ is the query embedding, and $\mathbf{z}_{r(e)} = \text{Encode}(x_{r(e)})$ is the embedding of the relation type associated with edge e computed by the same language encoder used in KG indexing, with $x_{r(e)}$ being the textual description of relation $r(e)$. Then we compute the MST by applying Kruskal’s algorithm [19]:

$$\mathcal{K}_4 = \text{MST}(G[\mathcal{V}_{\text{PPR}}]). \quad (5)$$

Kruskal’s algorithm greedily adds edges in increasing order of $\text{cost}(\cdot)$ while avoiding cycles, thereby connecting the subgraph with important entities via relations most relevant to the query.

3.4 Knowledge Preference Alignment

While the Mixture-of-Experts retrieval agent provides structured KG knowledge, the retrieved triples or subgraphs are misaligned with the natural-language inputs expected by LLM-based recommenders. Naive textual serialization can introduce noise and lose structural information, degrading recommendation performance. To address this issue, MixRAGRec introduces a knowledge preference alignment agent that converts retrieved KG knowledge into LLM-friendly representations while preserving recommendation-relevant information.

Let $\mathcal{K}$ denote the retrieved knowledge returned by the selected retrieval expert. Expert 2 returns a set of triples, Expert 3 returns a k -hop neighborhood subgraph, Expert 4 returns a connected subgraph, and Expert 1 corresponds to no retrieval. Since the $\mathcal{K}$ is structured, we first transform it into a stable textual draft using a carefully crafted template module $\text{Temp}(\cdot)$. To be more specific, for triple-level retrieval (i.e., Expert 2), $\text{Temp}(\cdot)$ converts each triple (h, r, t) into a short natural-language statement with relation-aware patterns, e.g., “*{head} has {relation} {tail}*”, and concatenates them into a compact list of knowledge statements. For graph-based retrieval (i.e., Expert 3 and Expert 4), $\text{Temp}(\cdot)$ linearizes the retrieved graph into a set of salient relational statements (e.g., edges along short paths among the retrieved entities), providing a consistent input format across retrieval granularities.

Given the query q and the templated text $\text{Temp}(\mathcal{K})$, the knowledge preference alignment agent further refines the draft into an aligned knowledge snippet $\tilde{\mathcal{K}} = \text{Align}_{\psi}(q, \text{Temp}(\mathcal{K}))$, where $\text{Align}_{\psi}(\cdot)$ is the knowledge preference alignment agent and trained to produce knowledge that is maximally useful to the downstream recommendation agent. The aligned knowledge $\tilde{\mathcal{K}}$ serves as a textual interface between structured KG knowledge and LLM-based recommendations and is appended to the downstream recommendation prompt along with the user’s query.

3.5 Preference-Optimized Recommendation via Contrastive Feedback

In knowledge-augmented recommendation, users’ contexts often contain numerous reasonable items that are highly similar to the

target item, such as items sharing the same genre, entities, or overlapping attributes. Such similar items can easily confuse the model, especially when external knowledge from the KG is integrated. Directly optimizing the recommender may fail to sufficiently differentiate the target items from these strong alternatives, reducing the recommendation performance. To this end, we introduce a contrastive learning-reinforced recommendation agent to optimize the recommendation agent with contrastive preference feedback.

Specifically, given a user’s query q and aligned knowledge $\tilde{\mathcal{K}}$, we define the recommendation context as $x = (q, \tilde{\mathcal{K}})$. Let $\mathcal{O}$ denote the candidate item set under context x , $o^* \in \mathcal{O}$ is defined as the target item and other items $o^- \in \mathcal{O} \setminus \{o^*\}$ are treated as negative items. The objective is to train the model to assign a higher probability to the positive item than to negative items. However, directly optimizing this objective is costly and less efficient due to the large scale of the action space. To solve this problem, we introduce hard negative sampling to select the top- N competitive negative items according to the current recommendation policy, encouraging the model to learn sharper distinctions among semantically similar items. Formally, we define the hard negative set as:

$$\mathcal{N}_{\text{hard}}(x) = \text{TopN}_{o \in \mathcal{O} \setminus \{o^*\}} \pi_{\omega}(o | x), \quad (6)$$

and construct preference pairs (x, o^*, o^-) with $o^- \in \mathcal{N}_{\text{hard}}(x)$. The training objective is then defined as:

$$\mathcal{L}_{\text{pref}} = -\log \sigma \left(\beta \left[\log \frac{\pi_{\omega}(o^* | x)}{\pi_{\text{ref}}(o^* | x)} - \log \frac{\pi_{\omega}(o^- | x)}{\pi_{\text{ref}}(o^- | x)} \right] \right), \quad (7)$$

where $\sigma(\cdot)$ is the sigmoid function and β is a temperature hyperparameter. We introduce π_{ref} as an anchoring baseline to stabilize learning, and optimize π_{ω} to assign higher probability to the target item. When multiple hard negatives are available, we compute Eq. (7) for each (o^*, o^-) pair and average the loss across negatives.

3.6 Mixture-of-Experts Multi-Agent Policy Optimization (MMAPO)

Due to the intricate interconnections among the three agents, it is not trivial to train MixRAGRec. Specifically, the recommendation performance depends on the selected retrieval expert, knowledge alignment quality, and the recommendation agent’s capabilities. These tight interconnections make it difficult to evaluate each agent’s influence during training. Therefore, vanilla optimization methods make the model tend to select the expert based on fixed preferences rather than selecting experts based on the query.

To this end, we propose Mixture-of-Experts Multi-Agent Policy Optimization (MMAPO), which coordinates the three agents under a shared objective and enables the model to identify the most suitable retrieval expert based on the user’s query, as shown in Figure 2. Specifically, MMAPO couples agents via a shared reward that combines recommendation performance with a marginal information gain. This design encourages flexible expert selection that balances knowledge richness and efficiency, while jointly improving knowledge alignment and recommendation performance.

3.6.1 Advantage estimation. In MixRAGRec, the retrieval expert selected by the Mixture-of-Experts retrieval agent affects the final reward only through the downstream aligned knowledge and the

subsequent recommendation. This delayed dependency makes the retrieval policy difficult to optimize from raw returns. Therefore, MMAPO adopts advantage-based credit assignment with learned value baselines.

We unfold the pipeline into a trajectory of decision steps, where the retrieval action and the alignment token generations are treated as actions at different steps. Let r_t denote the reward at step t and define the discounted return $G_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$, where $\gamma \in (0, 1]$ is the discount factor. We use a sparse terminal reward $r_t = 0$ for $t < T$ and $r_T = R_{\text{total}}$. MMAPO maintains value baselines for the retrieval and alignment agents. Specifically, the retrieval agent uses a value network $V_\phi(s_t)$ over its state s_t , and the alignment agent is equipped with a value head $V_\psi(c_t)$ defined on its generation context c_t (i.e., the prompt plus the generated prefix). Let $V(\cdot)$ denote the corresponding value baseline (i.e., $V(u_t) = V_\phi(s_t)$ for retrieval steps, and $V(u_t) = V_\psi(c_t)$ for alignment steps). We compute the temporal-difference residual: $\delta_t = r_t + \gamma V(u_{t+1}) - V(u_t)$, and estimate advantages using Generalized Advantage Estimation (GAE):

$$A_t^{\text{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad (8)$$

where $\lambda \in [0, 1]$ controls the bias-variance trade-off. This estimator provides low-variance learning signals and propagates downstream reward back to the retrieval decision and the alignment generation process through their respective value baselines.

3.6.2 Reward Design. To encourage flexible expert selection that balances recommendation performance and retrieval utility, we introduce a two-dimensional reward function, as follows:

$$R_{\text{total}} = R_{\text{rec}} + \alpha \cdot R_{\text{MIG}}, \quad (9)$$

where α is the weighting coefficient for coordinating the two reward dimensions. The recommendation reward R_{rec} measures the performance of the final recommendation:

$$R_{\text{rec}} = \begin{cases} \text{conf}(o), & \text{if } o = o^*, \\ -0.1, & \text{otherwise,} \end{cases} \quad (10)$$

where $\text{conf}(o) = \pi_\omega(o \mid q, \tilde{\mathcal{K}}, \mathcal{O})$ denotes the model confidence assigned to the recommended item.

To make expert selection cost-aware, we introduce a marginal information gain reward:

$$R_{\text{MIG}} = \Delta I(e) - \eta \cdot C(e), \text{ where } \Delta I(e) = D_{\text{KL}}(P_{\text{expert}} \parallel P_{\text{base}}), \quad (11)$$

where P_{expert} denotes the recommendation distribution when expert e is invoked, P_{base} is a baseline distribution (e.g., Expert 1 without retrieval), and $C(e)$ is a computational cost assigned to expert e , increasing with retrieval complexity. By rewarding information gain while penalizing retrieval cost, R_{MIG} encourages expensive retrieval only when it yields sufficient utility to justify the added cost. MMAPO optimizes the retrieval and alignment policies under this shared reward while training the recommendation agent with contrastive preference feedback. The core procedure is summarized in Algorithm 1, and the full training objective and update rules are provided in Appendix B.

Table 1: Basic statistics of three datasets and the KGs. “Items in KG” indicates the number of items that appeared in both the KG and the dataset.

Datasets	MovieLens-1M	MovieLens-20M	LFM-1K
User	6,040	138,287	992
Item	3,883	27,278	107,528
Interaction	1,000,209	20,000,263	19,150,868
Items in KG	3,253	24,700	33,736
Entities	24,840	72,673	180,457
Relations	168	223	312
KG triples	64,688	272,173	516,604

4 Experiments

In this section, we evaluate the effectiveness of MixRAGRec through extensive experiments. We will first introduce the experimental settings, including the datasets, compared baseline methods, evaluation metrics, and implementation details. Then, we report the main experimental results, highlighting the performance of MixRAGRec compared with various strong baselines. Finally, we conduct comprehensive additional analyses, including efficiency evaluation, ablation studies, and hyper-parameter analysis.

4.1 Datasets

We evaluate the performance of MixRAGRec on three widely used real-world recommendation datasets covering both movie and music domains. **MovieLens-1M**¹ contains approximately one million user-movie ratings, along with basic textual metadata such as movie titles. **MovieLens-20M**² is a large-scale movie dataset containing over 20 million ratings from more than 138,000 users for approximately 27,000 movies. **LastFM-1K**³ is a music dataset that records more than 19 million user listening histories and associated artist information. Following our framework design, we adopt DBpedia⁴ as the data source and construct corresponding knowledge graphs for each dataset. The statistics of the datasets and their corresponding KGs are summarized in Table 1.

4.2 Baselines

We compare MixRAGRec with a carefully constructed set of baselines to provide a comprehensive evaluation. These baselines cover (i) Zero-shot Inference, (ii) LLM-based Recommendation Models, and (iii) KG-RAG for LLM-based Recommendation Models.

Zero-shot inference. These baselines directly prompt LLMs such as *Gemini-2.0-Flash*, *GPT-4o* [17], *Deepseek-r1* [12], *LLaMA3-8B* [6], and *Mistral-7B* to perform recommendations without any task-specific fine-tuning or external knowledge augmentation.

LLM-based Recommendation Models. These methods adapt LLMs to recommendation tasks by fine-tuning on recommendation datasets. We include *TallRec* [2], which fine-tunes LLMs using instruction-tuning for recommendation, and *Rec-r1* [24], which further adapts LLMs for reasoning-augmented recommendation by reinforcement learning.

¹<https://grouplens.org/datasets/movielens/1m/>

²<https://grouplens.org/datasets/movielens/20m/>

³<http://ocelma.net/MusicRecommendationDataset/lastfm-1K.html>

⁴<https://www.dbpedia.org/>

Table 2: Performance comparison across LLM-based recommendation baselines and KG-RAG methods. Best results are in bold and the second-best results are underlined.

Model	MovieLens-1M			MovieLens-20M			LFM-1K		
	Accuracy	Recall@3	Recall@5	Accuracy	Recall@3	Recall@5	Accuracy	Recall@3	Recall@5
<i>Zero-shot Inference</i>									
Gemini-2.0-Flash	0.389	N/A	N/A	0.467	N/A	N/A	0.728	N/A	N/A
GPT-4o	0.420	N/A	N/A	0.524	N/A	N/A	0.685	N/A	N/A
Deepseek-r1	0.393	N/A	N/A	0.445	N/A	N/A	0.634	N/A	N/A
LLaMA3-8B _{zero-shot}	0.130	N/A	N/A	0.129	N/A	N/A	0.234	N/A	N/A
<i>LLM-based Recommendation Models</i>									
TallRec _{LLaMA3-8B} [2]	0.391	<u>0.663</u>	<u>0.779</u>	0.521	<u>0.781</u>	<u>0.868</u>	0.699	0.848	0.891
Rec-r1 _{LLaMA3-8B} [24]	0.402	0.558	0.634	0.594	0.761	0.815	<u>0.872</u>	<u>0.926</u>	<u>0.938</u>
<i>KG-RAG for LLM-based Recommendation Models</i>									
KG-Text _{LLaMA3-8B} [39]	0.234	N/A	N/A	0.301	N/A	N/A	0.545	N/A	N/A
KAPING _{LLaMA3-8B} [1]	0.232	N/A	N/A	0.274	N/A	N/A	0.530	N/A	N/A
G-retriever _{LLaMA3-8B} [14]	0.395	0.648	0.743	0.491	0.722	0.801	0.690	0.825	0.860
K-RagRec _{LLaMA3-8B} [36]	0.454	0.659	0.707	0.578	0.720	0.755	0.845	0.886	0.906
MixRAGRec_{LLaMA3-8B}(Ours)	0.504	0.798	0.882	0.676	0.900	0.940	0.934	0.990	0.992
Improvement	11.0%	20.4%	13.2%	13.8%	15.2%	8.3%	7.1%	6.9%	5.8%
<i>Other Backbone LLM</i>									
Mistral-7B _{zero-shot}	0.105	N/A	N/A	0.097	N/A	N/A	0.318	N/A	N/A
TallRec _{Mistral-7B} [2]	0.393	0.665	0.761	0.545	<u>0.809</u>	0.883	0.674	0.812	0.867
Rec-r1 _{Mistral-7B} [24]	0.338	0.433	0.490	0.505	0.599	0.648	0.362	0.477	0.552
KG-Text _{Mistral-7B} [39]	0.228	N/A	N/A	0.277	N/A	N/A	0.492	N/A	N/A
KAPING _{Mistral-7B} [1]	0.249	N/A	N/A	0.272	N/A	N/A	0.536	N/A	N/A
G-retriever _{Mistral-7B} [14]	0.372	0.641	0.775	0.542	0.758	0.846	0.763	0.856	0.890
K-RagRec _{Mistral-7B} [36]	<u>0.438</u>	<u>0.727</u>	<u>0.831</u>	<u>0.573</u>	0.801	<u>0.888</u>	<u>0.818</u>	<u>0.874</u>	<u>0.894</u>
MixRAGRec_{Mistral-7B}(Ours)	0.518	0.785	0.879	0.652	0.892	0.935	0.912	0.953	0.962
Improvement	18.3%	8.0%	5.8%	13.8%	10.3%	5.3%	11.5%	9.0%	7.6%

KG-RAG for LLM-based Recommendation Models. These baseline models augment LLMs by retrieving structured knowledge from the KG and are adapted in our experiments to support recommendation tasks. We include triple-level retrieval methods KG-Text [39] and subgraph-level retrieval methods KAPING [1] and G-retriever [14]. We also compare MixRAGRec with K-RagRec [36], designed for KG-RAG recommendation tasks, which retrieves knowledge subgraphs of different hops from KG for knowledge-augmented recommendation.

4.3 Evaluation Metrics

We evaluate recommendation performance using Accuracy and Recall@K ($K = 3, 5$). Accuracy measures whether the model’s recommendation matches the target item, while Recall@K measures whether the target item appears in the top-K recommended list. Following recent studies [36, 44], we select the item most recently interacted with by the user as the target item and use the 10 interactions preceding it as the user context. The model is then required to predict the user’s preferred item from a candidate pool consisting of the target item and 19 distractor items. For inference-based models that only output a single option without providing comparable scores over all candidates, we report Accuracy only.

4.4 Parameter Settings

Our framework is implemented in PyTorch, and all experiments are conducted on one NVIDIA H20-96G GPU. For the Mixture-of-Experts retrieval agent and knowledge preference alignment agent, we optimize the policy using a learning rate 3×10^{-4} , discount factor

$\gamma = 0.99$, GAE parameter $\lambda = 0.95$, clipping coefficient $\epsilon = 0.2$, and 4 update epochs per iteration. For preference optimization in the recommendation agent, we set the temperature parameter to $\beta = 0.2$ and employ hard negative sampling with $N = 10$ negatives. For the shared reward design, we set the MIG weight $\alpha = 0.2$ and the cost penalty $\eta = 0.005$. Notably, the knowledge preference alignment agent and the recommendation agent adopt the same LLM backbone architecture (e.g., LLaMA3-8B and Mistral-7B) with independent parameters. Additional implementation details appear in Appendix A.

4.5 Overall Performance Comparison

We compare the recommendation performance of MixRAGRec with various carefully constructed baselines under the LLM backbones LLaMA3-8B and Mistral-7B, on three datasets across three metrics. The experimental results are reported in the Table 2. From the comparison, we have the following key observations:

- Zero-shot prompting is insufficient for recommendation. Across three datasets, zero-shot inference consistently underperforms methods adapted to recommendation (e.g., TallRec/Rec-r1) and knowledge-augmented approaches. Although closed-source LLMs (e.g., GPT-4o and Deepseek-r1) achieved stronger zero-shot Accuracy than open-source backbones (e.g., LLaMA3-8B and Mistral-7B), they still lag considerably behind the best baselines and MixRAGRec, highlighting the limitation of direct prompting for recommendations.
- Retrieving external knowledge from the KG can substantially improve recommendation performance, while the benefit depends

Table 3: Efficiency comparison on MovieLens-1M with the LLaMA3-8B backbone. We report average retrieval time and end-to-end total time in seconds (s). ACC denotes Accuracy.

Methods	ACC	Retrieval Time (s)	Total Time (s)
KG-Text	0.234	0.311	2.974
KAPING	0.232	0.247	2.708
G-retriever	0.395	2.411	2.907
K-RagRec	0.454	2.867	3.776
Expert 1 (DirectGenerator)	-	0.000	0.042
Expert 2 (TripleRetriever)	0.488	0.066	1.827
Expert 3 (SubgraphRetriever)	0.491	0.212	2.103
Expert 4 (ConnectedGraphRetriever)	0.499	0.374	2.271
MixRAGRec	0.504	0.063	1.563

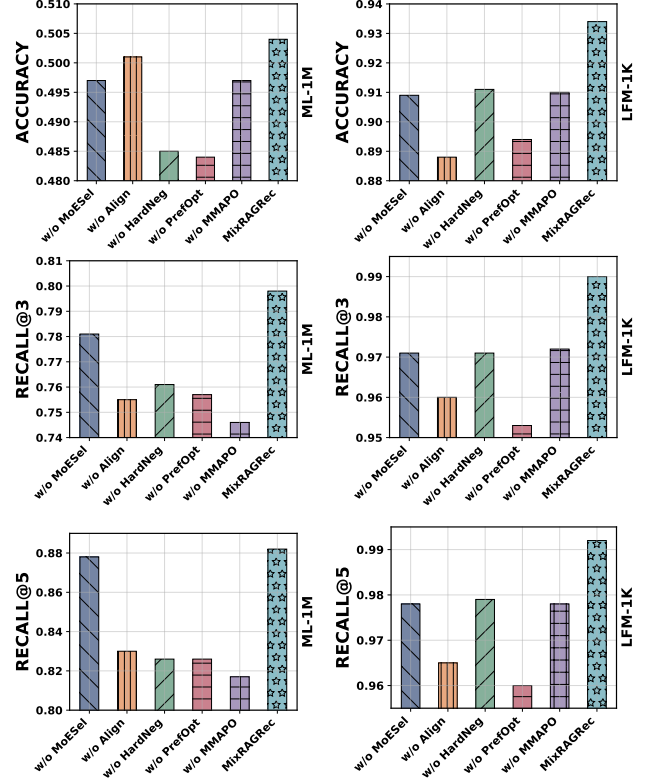
on how knowledge is retrieved and integrated. Directly injecting retrieved KG knowledge as text (e.g., KG-Text and KAPING) does not get consistent improvements over strong LLM-based recommender systems. In contrast, methods that leverage graph-level retrieval and representation (e.g., G-Retriever and K-RagRec) generally achieve stronger performance, suggesting that preserving structural information is important for effective knowledge augmentation.

- MixRAGRec outperforms the strongest baselines on all three datasets under both backbones, with relative improvements ranging from 5.8% to 20.4% under LLaMA3-8B and around 5.3% to 18.3% under Mistral-7B, highlighting the effectiveness of MixRAGRec. Notably, MixRAGRec also demonstrates significant improvement in the Recall@K metric, indicating that the proposed framework improves the quality of the top-K ranked recommendations rather than only the Accuracy.

4.6 Efficiency Study

To evaluate the efficiency of MixRAGRec, we measure the average retrieval time and end-to-end total latency on MovieLens-1M with the LLaMA3-8B backbone, using one NVIDIA H20-96G GPU. The results are summarized in Table 3. While lightweight retrieval methods like KG-Text and KAPING present fast retrieval efficiency, they have limited accuracy. Although graph-level retrieval methods (e.g., G-retriever and K-RagRec) achieve higher accuracy, they incur significantly higher retrieval time and total latency. In particular, K-RagRec exhibits the largest latency because it requires individual retrieval for each item. In contrast, MixRAGRec achieves the best accuracy while maintaining significantly lower retrieval time and total latency than baselines, suggesting that flexible expert selection can preserve the benefits of structured knowledge without unnecessary retrieval costs.

We further analyze different retrieval experts in MixRAGRec. As the retrieval granularity increases from Expert 2 to Expert 4, both retrieval time and accuracy increase, reflecting the expected cost–utility trade-off of richer KG knowledge. It is worth noting that MixRAGRec achieves the highest accuracy while keeping retrieval times close to the Expert 2, indicating that the learned policy invokes expensive retrieval only when it provides sufficient utility. These results highlight that MixRAGRec improves recommendation performance with lower cost, validating the effectiveness and efficiency of the proposed framework. We also analyze the retrieval complexity in Appendix C.

**Figure 4: Ablation results of MixRAGRec and five variants on MovieLens-1M and LFM-1K with the LLaMA3-8B backbone. We report Accuracy, Recall@3, and Recall@5.**

4.7 Ablation Study

We conduct ablation studies with LLaMA3-8B on MovieLens-1M and LFM-1K to assess the contribution of each key component in MixRAGRec. Specifically, we evaluate five ablated variants: (i) **w/o MoESel**, which replaces the Mixture-of-Experts selector policy with random expert selection; (ii) **w/o Align**, which removes the Knowledge Preference Alignment agent and uses only the templated verbalization Temp(K) as input to the recommender; (iii) **w/o HardNeg**, which replaces hard negative sampling with random negatives in contrastive preference training; (iv) **w/o PrefOpt**, which removes contrastive preference feedback for training the recommender; and (v) **w/o MMAPO**, which removes the shared objective and coordinated updates by dropping the MIG term and optimizing each agent separately with an accuracy-only reward using generic policy-gradient updates. The results are demonstrated in Figure 4.

Overall, the full MixRAGRec achieves the best performance across all metrics on both datasets, demonstrating the effectiveness of each component. Notably, the w/o Align variant resulted in the most significant performance degradation, especially on the LFM-1K dataset, suggesting that the Knowledge Preference Alignment agent is crucial for bridging the gap between retrieval structured knowledge and LLM inputs. In addition, removing hard negative sampling or contrastive feedback consistently degrades Accuracy and Recall@K on both datasets, highlighting the importance of

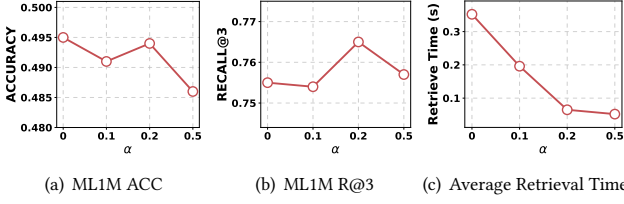


Figure 5: Effect of the MIG weight α on MovieLens-1M with the LLaMA3-8B backbone.

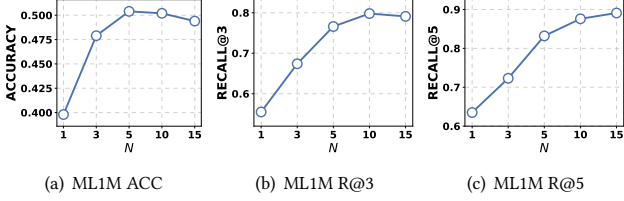


Figure 6: Effect of the number of hard negatives N on MovieLens-1M with the LLaMA3-8B backbone.

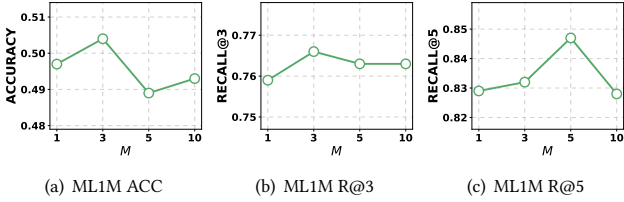


Figure 7: Effect of the retrieval budget M on MovieLens-1M with the LLaMA3-8B backbone.

explicitly separating the target item from highly competitive negatives in knowledge-augmented recommendation. Moreover, replacing the Mixture-of-Experts retrieval agent with random expert selection and removing MMAPO both lead to sub-optimal results, demonstrating the effectiveness of our proposed Mixture-of-Experts retrieval and optimization approach.

4.8 Hyper-parameter Analysis

In this sub-section, we study the impact of three key hyper-parameters of MixRAGRec on MovieLens-1M with LLaMA3-8B. Specifically, we evaluate the marginal information gain weight α , the number of hard negatives N , and the retrieval budget M .

4.8.1 Effect of the MIG weight α . We vary the MIG weight α to control the trade-off between retrieval utility and recommendation effectiveness in the shared reward. As shown in Figure 5, setting $\alpha = 0$ bases the shared reward solely on recommendation accuracy, resulting in the longest average retrieval time and relatively poorer ranking quality (Recall@3/5). Increasing α to a moderate value improves ranking performance while reducing retrieval time. In particular, $\alpha = 0.2$ achieves the best Recall@3/5 performance while significantly reducing the average retrieval time, suggesting that an appropriate MIG α encourages more cost-aware retrieval while guaranteeing the recommendation performance. When α is further increased to 0.5, retrieval efficiency improves slightly but performance is significantly reduced. Overall, these results highlight the importance of selecting an appropriate MIG weight to balance retrieval utility and recommendation effectiveness.

4.8.2 Effect of the number of hard negatives N . We analyze the impact of the number of hard negatives on recommendation performance on MovieLens-1M with the LLaMA3-8B backbone. Results are shown in Figure 6. Using too few hard negatives severely degrades performance. For example, when $N = 1$, both Accuracy and Recall@3/5 drop substantially, indicating that a single negative provides an insufficient contrastive signal. Increasing N to 5 and 10 improves both Accuracy and Recall@K, suggesting that additional hard negatives help the model better distinguish target items from competitive candidates. Further increasing N to 15 yields limited gains with trade-offs across metrics. Accordingly, we set $N = 10$ in our experiments.

4.8.3 Effect of the retrieval budget M . We study the effect of the retrieval budget M on MovieLens-1M with the LLaMA3-8B backbone. The budget M controls the retrieval scale across experts, and is instantiated differently for different granularities. For triple-level retrieval, we return the top- cM triples (with $c = 5$), while for subgraph-based retrieval, we use M to determine the number of retained nodes (or seed entities) before graph expansion. Results are shown in Figure 7. We observe that a small budget can miss useful knowledge, while a large budget may introduce noise and increase the burden on downstream alignment and utilization. On MovieLens-1M, $M = 3$ and $M = 5$ yield optimal Accuracy and Recall, respectively. Overall, it is important to select an appropriate retrieval budget based on the scale of the dataset and the KG.

5 Conclusion

In this work, we present MixRAGRec, a cooperative multi-agent framework for KG-RAG recommendation that improves both effectiveness and efficiency by enabling query-aware retrieval and knowledge utilization. Specifically, MixRAGRec includes a Mixture-of-Experts Retrieval Agent that selects among KG retrieval experts with different granularities, a Knowledge Preference Alignment Agent that converts structured KG knowledge into LLM-friendly text, and a Contrastive Learning-reinforced Recommendation Agent trained with contrastive preference feedback to better distinguish the target item from competitive candidates. To jointly train the three agents, we propose Mixture-of-Experts Multi-Agent Policy Optimization (MMAPO), which coordinates learning under a shared objective with recommendation feedback and marginal information gain. Experiments on three real-world datasets demonstrate the effectiveness and efficiency of MixRAGRec.

Acknowledgments

The research described in this paper has been partially supported by the Key Special Project of National Natural Science Foundation of China (Project No. 72442017), the General Research Funds from the Hong Kong Research Grants Council (project No. PolyU 15207322, 15200023, 15206024, and 15224524), Internal research funds from Hong Kong Polytechnic University (project no. P0059586, P0042693, P0048625, and P0051361), and Sheertek International (HK) Limited. This work was supported by computational resources provided by The Centre for Large AI Models (CLAIM) of The Hong Kong Polytechnic University.

- [44] Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024. Agentcf: Collaborative learning with autonomous language agents for recommender systems. In *Proc. World Wide Web Conf.* 3679–3689.
- [45] Yang Zhang, Fuli Feng, Jizhi Zhang, Keqin Bao, Qifan Wang, and Xiangnan He. 2025. Collm: Integrating collaborative embeddings into large language models for recommendation. *IEEE Trans. Knowl. Data Eng.* (2025).
- [46] Zihuai Zhao, Yajuan Ding, Wenqi Fan, and Qing Li. 2025. WebRec: Enhancing LLM-based Recommendations with Attention-guided RAG from Web. *arXiv preprint arXiv:2511.14182* (2025).
- [47] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2024. Recommender systems in the era of large language models (llms). *IEEE Trans. Knowl. Data Eng.* 36, 11 (2024), 6889–6907.
- [48] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.
- [49] Yaochen Zhu, Chao Wan, Harald Steck, Dawen Liang, Yesu Feng, Nathan Kallus, and Jundong Li. 2025. Collaborative Retrieval for Large Language Model-based Conversational Recommender Systems. In *Proc. World Wide Web Conf.* 3323–3334.

Appendix

A Implementation Details

The implementation details of MixRAGRec are summarized in Table 4. The first block reports the optimization-related hyper-parameters for the Mixture-of-Experts retrieval agent and the knowledge alignment agent, including the learning rate (3×10^{-4}), discount factor ($\gamma = 0.99$), GAE parameter ($\lambda = 0.95$), retrieval budget (Top- M with $M = 3$), clipping coefficient ($\epsilon = 0.2$), and training epochs. The optimization is set with batch size 64, buffer size 2048, and gradient clipping with max norm 0.5. The second block lists the preference optimization settings for the recommendation agent, including the temperature $\beta = 0.2$ and the number of hard negatives $N = 10$. The third block presents the shared reward hyper-parameters for marginal information gain, namely the MIG weight $\alpha = 0.2$ and cost penalty $\eta = 0.005$. The remaining blocks provide the parameter-efficient fine-tuning configuration (LoRA rank $r = 16$, scaling $\alpha_{\text{LoRA}} = 32$, dropout 0.1, and int8/fp16 settings) and the decoding/evaluation protocol (decoding temperature 0.8, candidate pool size 20, and Recall@ K with $K \in \{3, 5\}$). In addition, for KG indexing and similarity-based retrieval, we encode queries, entities, and triples into the same semantic space using a pre-trained sentence encoder, sentence-transformers/all-MiniLM-L6-v2. This encoder is used to build the entity and triple vector databases, and retrieval is conducted via Top- M similarity computation over the embedding space.

B MMAPO: Detailed Update Rules

With advantages from Eq. (8) and the shared reward in Eq. (9), MMAPO updates the three agents in a coordinated manner. As the retrieval and alignment agents affect the recommendation through sequential decisions, we adopt a clipped update rule to keep each policy update stable and prevent overly large changes between iterations. Specifically, for the Mixture-of-Experts retrieval agent and the knowledge preference alignment agent, we maximize the following objective:

$$\mathcal{J}_{\text{prox}}(\pi) = \mathbb{E} \left[\min \left(\rho_t A_t^{\text{GAE}}, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t^{\text{GAE}} \right) \right], \quad (12)$$

$$\rho_t = \exp(\log \pi(a_t | u_t) - \log \pi_{\text{old}}(a_t | u_t)),$$

where a_t is the action at step t (i.e., selecting an expert or generating a token), u_t denotes the corresponding input to the policy, and ϵ controls the update range.

For the contrastive learning-reinforced recommendation agent, we optimize the contrastive preference objective in Section 3.5, which encourages the model to assign a higher probability to the target item than to hard negatives. By updating the three agents, MMAPO enables MixRAGRec to learn when to retrieve, how to present KG knowledge, and how to leverage the aligned knowledge for recommendation. The core procedure of the MMAPO algorithm is presented in Algorithm 1.

Table 4: Hyper-parameter settings of MixRAGRec.

Item	Value
Policy Optimization (Retrieval & Alignment)	
learning rate	3×10^{-4}
discount factor γ	0.99
GAE parameter λ	0.95
Retrieval budget M	3
clipping coefficient ϵ	0.2
epochs	3
Preference Optimization (Recommendation)	
temperature β	0.2
number of hard negatives N	10
Shared Reward (MIG)	
MIG weight α	0.2
cost penalty η	0.005
Parameter-efficient Fine-tuning (LoRA)	
LoRA rank r	16
LoRA scaling α_{LoRA}	32
int8	True
fp16	True
LoRA dropout	0.1
Decoding & Evaluation Protocol	
decoding temperature	0.8
candidate pool size	20
evaluated Recall@ K	$K \in \{3, 5\}$

C Retrieval Complexity

We analyze the retrieval complexity of MixRAGRec, compared with two representative baselines. Table 5 summarizes the worst-case retrieval time complexity. Let M denote the retrieval budget (Top- M), L the user history length, and d the embedding dimension. We use $|K|$ to denote the size of returned structured knowledge (e.g., the number of triples/edges). G-retriever performs query-level retrieval by retrieving relevant nodes and triples/edges from their embedding spaces and then constructing a connected subgraph (e.g., via a Steiner-tree-style procedure), resulting in a cost of $O((|V| + |T|) \cdot (d + \log M) + |K|)$. In contrast, K-RagRec indexes multi-hop item-centered subgraphs in a single vector database and conducts similarity search for each item in the user history. Thus, its retrieval complexity scales linearly with the user history length L , i.e., $O(L \cdot (|S| \cdot (d + \log M) + |K|))$.

Our Mixed KG experts span different retrieval granularities with distinct costs. Expert 1 incurs no retrieval cost. Expert 2 performs Top- M triple retrieval over $\mathcal{T}$, costing $O(|T| \cdot (d + \log M))$. Expert 3 and Expert 4 first retrieve Top- M seed entities from $\mathcal{V}$ and

Algorithm 1 MixRAGRec (MMAPO Training)

Require: Training set $\mathcal{D}$ with instances (q, O, o^*) ; KG G ; retrieval experts $\mathcal{E}_{\text{exp}}$; retrieval policy π_θ with value V_ϕ ; alignment policy π_ψ with value head V_ψ ; recommender policy π_ω with reference π_{ref} ; hyper-parameters $\gamma, \lambda, \epsilon$ (advantage estimation and proximal update), α, η (MIG), β (preference update), N (hard negatives), retrieval budget M .

- 1: Initialize parameters $(\theta, \phi, \psi, \omega)$.
- 2: **for** each training iteration **do**
- 3: Sample a mini-batch $\mathcal{B} \subset \mathcal{D}$.
- 4: Initialize buffers $\mathcal{T}$ (trajectories) and $\mathcal{P}$ (preference pairs).
- 5: **for** each $(q, O, o^*) \in \mathcal{B}$ **do**
- 6: Construct retrieval state s from q .
- 7: Select retrieval expert $e \sim \pi_\theta(\cdot | s)$ and retrieve structured knowledge $\mathcal{K}_e$ from G with budget M .
- 8: Convert $\mathcal{K}_e$ into a stable textual draft $\tilde{\mathcal{K}} \leftarrow \text{Temp}(\mathcal{K}_e)$.
- 9: Generate aligned evidence $\tilde{\mathcal{K}} \sim \pi_\psi(\cdot | q, \tilde{\mathcal{K}})$.
- 10: Predict item distribution $P_{\text{expert}}(\cdot) \leftarrow \pi_\omega(\cdot | q, \tilde{\mathcal{K}}, O)$ and obtain output o .
- 11: Compute shared reward $R_{\text{total}} \leftarrow R_{\text{rec}} + \alpha \cdot R_{\text{MIG}}$, where R_{rec} measures recommendation correctness/confidence and R_{MIG} measures marginal information gain with a retrieval cost penalty (using Expert 1 as the baseline to form P_{base}).
- 12: Store the coupled trajectory (states/contexts and actions for expert selection and alignment token generation) into $\mathcal{T}$ with terminal reward R_{total} .
- 13: Mine hard negatives $\mathcal{N}_{\text{hard}} \leftarrow \text{TopN}_{o' \in O \setminus \{o^*\}} \pi_\omega(o' | q, \tilde{\mathcal{K}}, O)$.
- 14: Add preference pairs (x, o^*, o^-) into $\mathcal{P}$ for each $o^- \in \mathcal{N}_{\text{hard}}$, where $x = (q, \tilde{\mathcal{K}})$.
- 15: **end for**
- 16: **Advantage estimation.**
- 17: Estimate advantages for retrieval using $V_\phi(s)$ and for alignment using $V_\psi(c)$ (Eq. 8).
- 18: **Coordinated policy update (MMAPO).**
- 19: Update the retrieval selector π_θ and alignment policy π_ψ via a proximal clipped policy update driven by the estimated advantages.
- 20: Update the recommender π_ω via preference-based contrastive optimization on $\mathcal{P}$ with reference π_{ref} .
- 21: **end for**
- 22: **return** Trained parameters $(\theta^*, \psi^*, \omega^*)$.

then expand/construct subgraphs. Therefore, the complexity is $O(|\mathcal{V}| \cdot (d + \log M) + |K_e|)$ for $e \in \{3, 4\}$, where $|K_e|$ is linear in the size of the returned subgraph.

Importantly, MixRAGRec does not commit to a single retrieval strategy. Instead, it flexibly routes each query to an appropriate expert, enabling a query-dependent trade-off between retrieval cost and knowledge utility. As a result, MixRAGRec can avoid unnecessary heavy retrieval for simple queries while still invoking richer subgraph retrieval when needed, yielding a favorable accuracy-efficiency balance compared with baselines that rely on a fixed retrieval pipeline or perform retrieval repeatedly along the user history.

D Expert Selection Distribution

We further analyze the routing behavior of the Mixture-of-Experts retrieval agent under both standard and cold-start settings. Figure 9 reports the expert selection distribution on LFM-1K with the Mistral-7B backbone. The agent allocates most instances to Expert 1–3,

Table 5: Comparison of retrieval time complexity. M is the retrieval budget (Top- M) and L is the user history length.

Method	Online retrieval time complexity
G-retriever	$O((\mathcal{V} + \mathcal{T}) \cdot (d + \log M) + K)$
K-RagRec	$O(L \cdot (S \cdot (d + \log M) + K))$
Expert 1 (DirectGenerator)	$O(1)$
Expert 2 (TripleRetriever)	$O(\mathcal{T} \cdot (d + \log M))$
Expert 3 (SubgraphRetriever)	$O(\mathcal{V} \cdot (d + \log M) + K_e), e = 3$
Expert 4 (ConnectedGraphRetriever)	$O(\mathcal{V} \cdot (d + \log M) + K_e), e = 4$
MixRAGRec	depends on the selected expert above

Table 6: Cold-start recommendation performance comparison with the strongest KG-RAG-enhanced LLM recommendation methods on LLaMA3-8B across three metrics. The best performances are labeled in bold.

Methods	Accuracy	Recall@3	Recall@5
LLaMA3-8B _{zero-shot}	0.083	N/A	N/A
G-retriever	0.356	0.641	0.738
K-RagRec	0.392	0.706	0.748
MixRAGRec	0.501	0.778	0.846

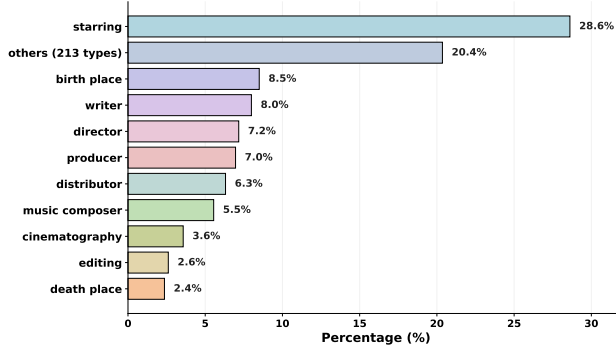
with proportions of 30.2%, 28.6%, and 28.2%, respectively, while Expert 4 is selected less frequently (13.0%). This pattern indicates that, for LFM-1K, the selector predominantly relies on lightweight or moderate-granularity retrieval and invokes the most expensive expert only for a smaller subset of instances.

We also examine the selection distribution in the cold-start scenario. As shown in Figure 10, MixRAGRec chooses Expert 3 for 37.9% of instances and Expert 4 for 58.7% of instances, suggesting that cold-start queries more often benefit from higher-granularity relational knowledge. These results highlight that MixRAGRec adapts retrieval granularity to different data contexts rather than using a fixed strategy.

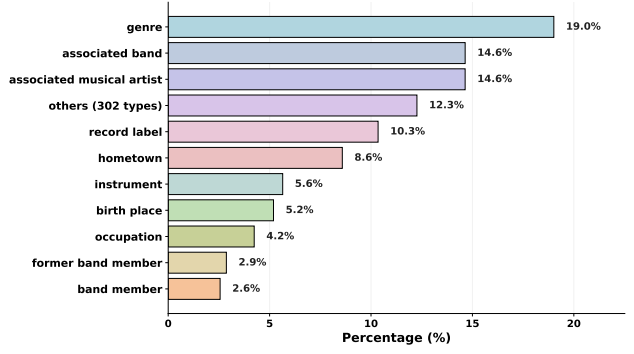
E Cold-start Recommendation Study

Cold-start recommendation remains an important challenge in practice. To evaluate MixRAGRec under this setting, we compare it with strong KG-RAG-enhanced baselines on MovieLens-1M using the LLaMA3-8B backbone, as reported in Table 6. Zero-shot prompting performs poorly in the cold-start scenario, which further motivates grounding recommendations with RAG. Compared with other KG-RAG baselines, MixRAGRec achieves the best results and remains robust under the cold-start setting, demonstrating the effectiveness of our framework.

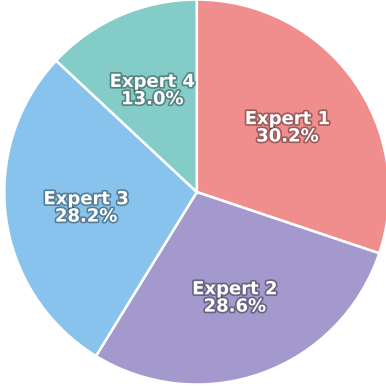
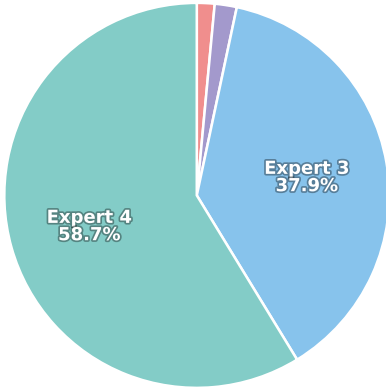
We further examine the expert-selection behavior in cold-start scenarios. Figure 10 shows that MixRAGRec selects subgraph-level retrieval (Expert 3) for 37.9% of queries and connected-graph retrieval (Expert 4) for 58.7% of queries, suggesting that cold-start instances frequently require richer relational context. This trend aligns with the gains in Table 6 and highlights the effectiveness of MixRAGRec in adapting retrieval granularity to the cold-start recommendations.



(a) KG for MovieLens-20M



(b) KG for LFM-1K

Figure 8: Relation-type distributions of the constructed KGs for (a) MovieLens-20M and (b) LFM-1K.**Figure 9: Expert selection distribution of the Mixture-of-Experts retrieval agent on LFM-1K with Mistral-7B.****Figure 10: Expert selection distribution of the Mixture-of-Experts retrieval agent on cold-start recommendation scenario.**

F More Related Work

Large language models (LLMs) have recently emerged as a promising paradigm for recommendation due to their strong semantic

understanding and generalization capabilities. Early research reformulates recommendations as a language modeling task to leverage LLMs’ language understanding and in-context learning capabilities for user preference modeling [2, 10, 25]. For example, LLMRank [15] employs instruction-based prompts constructed from historical interactions to perform item re-ranking, whereas TALLRec [2] introduces an instruction-tuning framework that directly recommends target items by modeling user interest from text-enriched historical interactions (e.g., item titles). To further enhance performance, subsequent efforts incorporate collaborative signals into LLM-based recommendations. Specifically, LLaRa [22] and CoLLM [45] treat collaborative information as an additional modality and align it with the LLM token space through learned mappings. Moreover, tokenization-based approaches encode external signals into item identifiers, representing items as discrete semantic or collaborative indices to facilitate LLM-based recommendation [34, 48]. Despite these advancements, LLM-based recommender systems remain constrained by static parametric knowledge and have limitations in obtaining up-to-date information. Therefore, to avoid costly fine-tuning, it is important to introduce RAG to incorporate up-to-date external knowledge.

G Knowledge Graph Statistics

To support the KG-RAG recommendations, we construct an item-centric knowledge graph (KG) for each dataset. Table 1 reports basic statistics of the three real-world datasets and their associated KGs, including the numbers of users, items, interactions, and the aligned items that appear in both the dataset and the KG (*Items in KG*). Figure 8 further visualizes the relation-type distributions for the MovieLens-20M and LFM-1K KGs. Overall, the KG coverage is substantial across datasets, providing rich, structured knowledge for grounding LLM-based recommendations. In addition, the KGs exhibit diverse schema complexity, with hundreds of relation types and up to hundreds of thousands of triples, which motivates retrieval designs that can flexibly control the granularity and amount of retrieved knowledge.

H Prompts used in MixRAGRec

We present the prompts for our MixRAGRec in Table 7 and 8. Specifically, the prompt used for Knowledge Preference Alignment Agent is shown in Tables 7, and the prompt used for Contrastive Learning-reinforced Recommendation Agent is shown in Tables 8.

I Case Study

To provide an intuitive understanding of how MixRAGRec operates in practice, we present three representative instances from MovieLens-1M and LFM-1K in Tables 9, 10, and 11. All cases follow the same pipeline: MixRAGRec first routes the input instance to a retrieval expert, optionally retrieves structured KG knowledge, aligns the knowledge into concise natural language, and finally produces a recommendation over the candidate pool.

Table 9 shows a MovieLens-1M example routed to **Expert 2 (TripleRetriever)**. The retrieved triples highlight significant relations related to the user’s recent interests, and the alignment agent converts them into a short, fluent knowledge snippet directly consumable by the recommendation agent. Conditioned on this aligned knowledge, the model assigns a highly peaked probability mass to the correct option, illustrating how lightweight triple-level retrieval can be sufficient when a small number of key facts already distinguish the target item.

Table 10 presents a MovieLens-1M instance where the selector chooses **Expert 3 (SubgraphRetriever)**. Compared with isolated triples, subgraph-level retrieval provides richer relational context. In this example, the retrieved subgraph captures a compact family-oriented cluster, and the alignment agent converts it into an LLM-friendly description for downstream recommendation. Conditioned on the aligned knowledge, the recommendation agent ranks the ground-truth option highest, illustrating the benefit of subgraph-level evidence when a multi-relation context is needed.

Finally, Table 11 shows an LFM-1K instance routed to **Expert 1 (DirectGenerator)**, where MixRAGRec skips retrieval and alignment entirely. The listening history already provides strong textual cues for the backbone model, and bypassing retrieval avoids unnecessary overhead while still yielding a confident correct prediction. This case highlights that MixRAGRec can preserve efficiency by not invoking retrieval when external evidence is unlikely to offer additional benefit.

Overall, these cases reflect the key advantage of MixRAGRec in controlling the effectiveness–utility trade-off. It can rely on lightweight evidence when it is sufficient (Table 9), avoid over-retrieval that may introduce noise (Table 10), and skip retrieval altogether for easy instances (Table 11).

Table 7: General prompt template for the Knowledge Preference Alignment Agent.

Knowledge Preference Alignment Agent Prompt
Shared Input
User Query: Watching history: “Movie A”, “Movie B”, ..., “Movie J”. User Preferences: general user Domain: Movie/Music History Label: "watching history"/ "listening history" Item Type: "movie"/"artist"
Expert 2 (Triple-level Retrieval)
Task Description: Convert the following factual triples into natural language knowledge suitable for movie recommendations. Retrieved Triples: {structured_triples_from_KG} Instructions: (1) Extract relevant facts from the triples (2) Convert to natural fluent text (3) Focus on genres, themes, directors, and user preferences
Expert 3 (Neighborhood Subgraph)
Task Description: Transform the following knowledge graph subgraph into natural language suitable for movie recommendations. Retrieved Subgraph: {structured_subgraph_from_KG} Instructions: (1) Identify key entities and relationships (2) Describe connection patterns (3) Highlight genres, themes, and stylistic similarities (4) Keep description concise and relevant
Expert 4 (Connected Subgraph)
Task Description: Transform the following connected knowledge graph into natural language for movie recommendations. Retrieved Connected Graph: {pagerank_mst_graph_from_KG} Instructions: (1) Extract core themes from connected entities (2) Describe shared characteristics (3) Focus on genres, directors, and cinematography (4) Provide a coherent narrative summary
Output
Knowledge Summary: {aligned_knowledge_summary}

Table 8: General prompt template for the Contrastive Learning-reinforced Recommendation Agent.

Contrastive Learning-reinforced Recommendation Agent Prompt
Shared Input
Domain: Movie/Music History Label: "watching history"/ "listening history" Item Type: "movie"/"artist"
Default (with KG knowledge)
Instruction: Based on the user’s watching history, select the best movie from options A-T. User Query: Watching history: “Movie A”, “Movie B”, ..., “Movie J”. Knowledge: {aligned_knowledge_summary} Options: “A: Movie”, “B: Movie”, ..., “T: Movie”. Select a movie from options A to T that the user is most likely to be interested in.
For Expert 0 (no KG knowledge)
Instruction: Based on the user’s watching history, select the best movie from options A-T. User Query: Watching history: “Movie A”, “Movie B”, ..., “Movie J”. Options: “A: Movie”, “B: Movie”, ..., “T: Movie”. Select a movie from options A to T that the user is most likely to be interested in.

Table 9: Case study on Movie with Expert 2 (TripleRetriever).

Case Study: Movie (TripleRetriever)
(1) Input instance. User Query (watching history): “Parenthood”, “Dark City”, “Reservoir Dogs”, “Aliens”, “The Sixth Sense”, “The Terminator”, “A Simple Plan”, “Die Hard”, “Seven”, “The Long Kiss Goodnight”. Candidate options (A–T): A: Hellbound: Hellraiser II, B: Bring It On, C: Escape from L.A., D: One Tough Cop, E: Boxing Helena, F: Separation, The, G: From the Journals of Jean Seberg, H: Terminator 2: Judgment Day, I: Following, J: The Big Country, K: One Flew Over the Cuckoo’s Nest, L: Grandview, U.S.A., M: Cotton Mary, N: Smiling Fish and Goat on Fire, O: School Daze, P: Soldier’s Daughter Never Cries, A, Q: Weekend at Bernie’s, R: Love and Death on Long Island, S: Roseanna’s Grave, T: Maurice. Ground-truth: H.
(2) Mixture-of-Experts routing. Selected expert: Expert 2 (TripleRetriever).
(3) KG retrieval output (structured). Retrieved triples: • The Terminator $\xrightarrow{\text{director}}$ James Cameron • Terminator 2: Judgment Day $\xrightarrow{\text{is}}$ highest-grossing film list • Indiana Jones and the Temple of Doom $\xrightarrow{\text{star}}$ Kate Capshaw • ...
(4) Knowledge Preference Alignment (Agent). Alignment prompt (abridged): Convert the following factual triples into natural language knowledge suitable for movie recommendations. Aligned knowledge (output): <i>Terminator 2: Judgment Day is a movie highly relevant to user queries, as its lead actor is associated with the query and it is related to other related films such as The Terminator. The Terminator, a movie starring Arnold Schwarzenegger, is a highly relevant film related to the user’s query ...</i>
(5) Recommendation (Contrastive Learning-reinforced Rec Agent). Output: H. Terminator 2: Judgment Day. Model confidence / option distribution (top): • H (Terminator 2): 0.92 $\Leftarrow$ (ground truth) • J (The Big Country): 0.01 • M (Cotton Mary): 0.01 • N (Smiling Fish and Goat on Fire): 0.01

Table 10: Case study on Movie with Expert 3 (SubgraphRetriever).

Case Study: Movie (SubgraphRetriever)
(1) Input instance. User Query (watching history): “The Lion King”, “The Road to El Dorado”, “Oliver & Company”, “The Incredible Journey”, “Home Alone”, “Jumanji”, “Flubber”, “Anastasia”, “Labyrinth”, “Tom and Huck”. Candidate options (A–T): A: Brassed Off, B: Mighty Ducks, The, C: Porky’s II: The Next Day, D: Ninotchka, E: The City, F: The Grass Harp, G: Like Water for Chocolate, H: Night of the Comet, I: Shaft, J: The Kid, K: Leave It to Beaver, L: Inspector Gadget, M: Fright Night Part II, N: Chopping Mall, O: Henry: Portrait of a Serial Killer, Part 2, P: Austin Powers: International Man of Mystery, Q: Romancing the Stone, R: NeverEnding Story II: The Next Chapter, S: Men in Black, T: Re-Animator. Ground-truth: L.
(2) Mixture-of-Experts routing. Selected expert: Expert 3 (SubgraphRetriever).
(3) KG retrieval output (structured). Retrieved subgraph: Entities: {Flubber (Movie), Jumanji (Movie), Oliver & Company (Movie)}. Relations: • Flubber $\xrightarrow{\text{genre}}$ family comedy • Flubber $\xrightarrow{\text{theme}}$ playful invention / slapstick • Jumanji $\xrightarrow{\text{genre}}$ family adventure • Jumanji $\xrightarrow{\text{theme}}$ fantasy adventure / kid-friendly • Oliver & Company $\xrightarrow{\text{genre}}$ animated family film • Oliver & Company $\xrightarrow{\text{theme}}$ lighthearted, for all ages • ...
(4) Knowledge Preference Alignment (Agent). Alignment prompt (abridged): Transform the following knowledge graph subgraph into natural language suitable for movie recommendations. Aligned knowledge (output): <i>The subgraph contains 3 core entities and a compact set of relations, where Flubber receives the highest relevance score and serves as the central node. The subgraph connects these entities through recurring family-oriented signals and lightweight entertainment attributes: For Flubber, the evidence emphasizes a comedy-focused profile with playful, kid-friendly cues; for Jumanji, the relations highlight family adventure elements and a fantasy-leaning storyline ...</i>
(5) Recommendation (Contrastive Learning-reinforced Rec Agent). Output: L Inspector Gadget. Model confidence / option distribution (top): • L (Inspector Gadget): 0.55 $\Leftarrow$ (ground truth) • B (Mighty Ducks): 0.22 • S (Men in Black): 0.09 • J (The Kid): 0.05

Table 11: Case study on Music with Expert 1 (DirectGenerator).

Case Study: Music (DirectGenerator)
(1) Input instance. User Query (listening history): “Boards Of Canada”, “Zazen Boys”, “Reclouse”, “Flying Lotus”, “The Ananda Project”, “Afterlife”, “Onra & Quetzal”, “Ian O’Brien”, “Onra & Quetzal”, “Reclouse”. Candidate options (A–T): A: <i>Dumptruck</i>, B: <i>Dj Lol</i>, C: <i>The Livids</i>, D: <i>Hei</i>, E: <i>Tactics</i>, F: <i>Onra & Quetzal</i>, G: <i>Eric Smax & Terri B.</i>, H: <i>Dan Curtin</i>, I: <i>Pebbles</i>, J: <i>Jochen Trappe</i>, K: <i>Versbox</i>, L: <i>Hans Zimmer</i>, M: <i>Melanie Doane</i>, N: <i>Ana D</i>, O: <i>Dave The Hustler</i>, P: <i>Jo O’Meara</i>, Q: <i>Codes In The Clouds</i>, R: <i>Fatal Bazooka</i>, S: <i>Only Fools And Horses</i>, T: <i>Pell Mell</i>. Ground-truth: F.
(2) Mixture-of-Experts routing. Selected expert: Expert 1 (DirectGenerator).
(3) KG retrieval output (structured). Retrieved knowledge: <i>Skipped</i> (DirectGenerator uses no KG retrieval).
(4) Knowledge Preference Alignment (Agent). Aligned knowledge (output): <i>Skipped</i> (no retrieved knowledge; $\tilde{\mathcal{K}} = \emptyset$).
(5) Recommendation (Contrastive Learning-reinforced Rec Agent). Output: F Onra & Quetzal. Model confidence / option distribution (top): • F (Onra & Quetzal): 0.76 $\Leftarrow$ (ground truth) • L (Hans Zimmer): 0.12 • P (Jo O’Meara): 0.06 • K (Versbox): 0.03